

Lex (.l) File Revision Sheet

This is your ultimate go-to guide for your Compiler Design Viva regarding the Lex file. Read through this to refresh your memory on all the core concepts we discussed.

1. The Big Picture: What is Lex?

Lex is a tool used for **Lexical Analysis**. It reads an input stream of characters (your source code) and converts them into meaningful chunks called **tokens** (like `IF`, `ID`, `ADDOP`). These tokens are then sent to the parser (Yacc/Bison) for syntax analysis.

Note: Lex works by using **Regular Expressions** to match patterns. When a pattern is matched, it executes the corresponding C++ code (called an action) to generate a token.

2. The 3 Sections of a Lex File

Every `.l` file is divided into three parts, separated by `%%`:

1. Declarations & Definitions Section

- **C++ Code (`%{ ... %}`):** Where you include libraries, declare global variables (like `line_num`, `outlog`), and define `YYSTYPE`.
- **Regular Definitions:** Where you create "aliases" for regex patterns to keep your code clean (e.g., `delim [ \t\v\r ]`).

2. Rules Section (Between `%%` and `%%`)

- The heart of the file. It follows the format: `pattern { action }`.
- Whenever Lex finds the `pattern`, it executes the C++ `action` inside the braces.

3. User Code Section (After the last `%%`)

- Optional C++ code, usually the `main()` function or `yywrap()`. (In your lab, `main` is handled in the `.y` file).

3. Important Built-in Variables and Options

%option noyywrap

Normally, when Lex hits the End-Of-File (EOF), it calls `yywrap()` to ask, "Is there another file to read?". By adding this option, you tell Lex: *"Assume there is only one file. Stop when you hit EOF."*

yytext

A built-in variable (char pointer) that holds the **exact string (lexeme)** that was just matched by the regular expression.

Example: If the rule `if` matches the word "if", `yytext` contains `"if"`. If the rule `[0-9]+` matches "123", `yytext` contains `"123"`.

4. How Lex Talks to Yacc (YYSTYPE & yylval)

Crucial Viva Question!

When Lex matches a pattern, it doesn't just return a token ID (like `ADDOP`). It also passes the *value* of that token to Yacc using a global variable named `yylval`.

By default, Yacc thinks `yylval` is just an integer. Here is how your code overrides this to pass complex data:

1. `#define YYSTYPE symbol_info*` : You override Yacc's stack type. You tell Yacc that `yylval` is now a pointer to your custom `symbol_info` class.
2. `symbol_info *s = new symbol_info((string)yytext, "ADDOP");` : When Lex matches a `+` or `-`, you create a new `symbol_info` object in memory. You store the exact lexeme (from `yytext`) inside it.
3. `yylval = (YYSTYPE)s;` : You point the global `yylval` variable to this new object.
4. `return ADDOP;` : You send the token ID to Yacc. When Yacc receives `ADDOP`, it looks inside `yylval` and finds your object, which tells it whether the `ADDOP` was specifically a `+` or a `-`.

5. Regular Expression Crash Course

Quantifiers (How many times?)

- **+** : One or more times. (Must appear at least once).
- ***** : Zero or more times. (Completely optional, but can repeat infinitely).
- **?** : Zero or one time. (Makes the preceding character or group **strictly optional**).

Sets and Groups

- **[...]** : Matches exactly ONE character from inside the brackets. (e.g., **[abc]** matches 'a', 'b', or 'c').
- **(...)** : Groups characters together so you can apply quantifiers to the whole group.

6. Deep Dive into Your Lab's Regexes

Whitespace (**delim** and **ws**)

```
delim    [ \t\v\r]
ws       {delim}+
```

- **delim** : Matches exactly one space, horizontal tab, vertical tab, or carriage return.
- **ws** : Matches **one or more** of those delimiters consecutively.
- **{ws} { /* ignore whitespace */ }** : When Lex matches spaces, it executes the C++ code inside the braces. Since it's just a comment, nothing executes, and Lex silently drops the space.

Newline

```
newline \r?\n
```

- **\n** : The standard newline character.
- **\r?** : An optional carriage return (handles Windows line breaks).

The Infamous **const_float**

```
{digit}*\. {digit}+ ([eE] [+ -]? {digit}+)? | {digit}+ [eE] [+ -]? {digit}+
```

This accepts two types of floating-point numbers, separated by the OR operator (**|**).

- **Part 1: Standard Decimals (3.14 , .5 , 2.5e-3)**

`{digit}*` (Zero or more digits) followed by `\.` (literal dot), then `{digit}+` (One or more digits). Finally, `([eE][+-]?{digit}+)?` makes the whole scientific notation part optional.

- **Part 2: Integers with Exponents (10e2 , 5E-4)**

`{digit}+` (One or more digits) followed directly by `[eE]` , an optional sign `[+-]?` , and the exponent digits `{digit}+` . No decimal needed!

7. The Token Generation & Logging Code

Whenever Lex matches a pattern, you run C++ code to log it and send a token to Yacc.

Scenario A: Simple Keywords

```
if
{
    loglist="Line no "+to_string(line_num)+" : Token <IF> Lexeme "+yytex
    outlog << loglist;
    return IF;
}
```

- `to_string(line_num)` : Converts the integer line count to a string for logging.
- `loglist = ...` : Builds the formatted string containing the `yytext` lexeme.
- `outlog << loglist;` : Writes the string into `22201161_log.txt` .
- `return IF;` : Returns the token ID to Yacc so parsing can continue.

Scenario B: Tokens that share an Object (Operators, IDs, Constants)

For things like operators or IDs, logging isn't enough. Yacc needs to know *exactly* what was typed.

```
"+" | "-"
{
    symbol_info *s = new symbol_info((string)yytext, "ADDOP");
    yylval = (YYSTYPE)s;

    loglist="Line no "+to_string(line_num)+" : Token <ADDOP> Lexeme "+yy
    outlog << loglist;
```

```
    return ADDOP;  
}
```

- **Why create an object?** Because `+` and `-` both return the exact same token ID: `ADDOP`.
- **The Object (`symbol_info *s`):** We dynamically allocate (`new`) a `symbol_info` object in memory. We pass it `yytext` (which contains either `+` or `-`) and the type `"ADDOP"`.
- **`yylval = (YYSTYPE)s;`** : We give this object to `yylval`. Now, when Yacc receives the `ADDOP` token, it can look inside `yylval` to retrieve the object and check if the user

actually typed a `+` or a `-` !

Viva Strategy: Speak confidently! If they ask about `yytext`, don't just say "it's the text." Say, "It's a built-in character pointer that holds the exact lexeme that triggered the current rule." You've got this! Good luck!